

Sıralama

CEN207 Veri Yapıları — Hafta 10

Dr. Öğr. Üyesi Uğur CORUH · 2026-2027 Güz

Bugünün planı (3 saat)

Saat	Konu
1	Maliyet modeli · kabarcık, seçmeli, eklemeli, shell sıralaması
2	Birleştirmeli sıralama (yukarıdan aşağı, aşağıdan yukarı) · hızlı sıralama (Lomuto, Hoare, en kötü durum)
3	Sayma, radix, kova sıralaması · kararlılık · deneysel karşılaştırma · seçim

Öğrenme çıktıları: ÖÇ.1 (temel veri yapılarını açıklama) · ÖÇ.2 (karmaşıklık analizi) · ÖÇ.7 (doğru yapıyı seçme)

Bu haftanın kavramları — nerede

Kavram	Nerede
Kabarcık, seçmeli, eklemeli, shell sıralaması	Bölüm 2–5
Birleştirmeli sıralama (yukarıdan aşağı / aşağıdan yukarı)	Bölüm 6
Hızlı sıralama (Lomuto / Hoare / en kötü durum)	Bölüm 7
Sayma, radix, kova sıralaması	Bölüm 8–10
Kararlılık, karşılaştırma, seçim	Bölüm 11–13

Kod örnekleri nasıl çalışır

- Her algoritmanın tam bir **C** ve **Java** programı var
- C: `gcc -std=c11 -Wall -Wextra -o /tmp/x file.c && /tmp/x`
- Java: `javac -d /tmp/j File.java && java -cp /tmp/j File`
- Kaynaklar: `code/week-10/c/` ve `code/week-10/java/`
- Her programın beklenen çıktısı hafta notlarında

Hatırlatma — diziler ve Big-O (Hafta 1)

- Bugünkü her sıralama düz bir dizi üzerinde çalışır, $O(1)$ erişim
- En iyi / ortalama / en kötü durum süsleme değil
- On bir farklı sıralamanın *neden* var olduğunun nedeni bu
- Aynı girdinin çığınca farklı maliyetler verdiğini izleyin

Hatırlatma — öbek sıralaması (Hafta 4)

- Zaten $O(n \log n)$, yerinde bir sıralama kurdunuz
- Bugün boyunca bir ölçüt olarak tutun
- Birleştirmeli sıralama garantisini eşler, $O(n)$ ek bedelle
- Hızlı sıralama onu yalnızca **ortalamada** eşler

İki gerçekten yeni fikir

- **Böl ve yönet:** böl, her yarıyı çöz, birleştir
- Birleştirmeli ve hızlı sıralamayı güçlendirir (Bölüm 6–7)
- **Karşılaştırmasız sıralama:** iki anahtarı asla karşılaştırma
- Sayma, radix, kova sıralamasını güçlendirir (Bölüm 8–10)

Bu haftanın haritası — bir bakışta

$O(n^2)$ ailesi	Garantili $O(n \log n)$
Kabarcık, seçmeli, eklemeli, shell	Birleştirmeli (yukarıdan aşağı, aşağıdan yukarı)
Ortalama $O(n \log n)$	Karşılaştırmasız, $O(n+k)$
Hızlı sıralama (Lomuto, Hoare)	Sayma, radix, kova sıralaması

Mini soru

Bugünden önce karşılaştığınız her sıralama, sıranın nasıl belirlendiği hakkında bir şey varsayıyor. Nedir bu, ve bugünkü hangi üç sıralama bu kuralı bozar?

Yanıt

İki anahtarı < ya da > ile karşılaştırmak.

Sayma, radix ve kova sıralaması (Bölüm 8–10)

anahtarları asla karşılaştırmaz — konumları doğrudan hesaplar.

1. Sıralama Problemi, ve Onu Ölçme Yöntemi

Başlangıç sorusu

Sıralama, bilgisayarlık tarihinde düzinelerce farklı şekilde çözüldü. Problem bu kadar basit görüldüğüne göre, neden bu kadar çok yanıtı ihtiyacı var?

Sıralama problemi, yeniden ifade edilmiş

- n değerlik dizi, bir kural (" a , b 'den küçük mü?")
- Her eleman bir sonrakine $\leq$ olacak şekilde yeniden düzenle
- Özgürlük: ne kadar ek bellek, neyi kullanıyorsunuz
- Özgürlük: eşit elemanların sırası değişebilir mi

Maliyeti ölçmek: iki sayı

Miktar	Sayar	Önemli olduğunda
karşılaştırmalar	her <code><</code> / <code>></code> / <code>==</code>	klasik maliyet ölçütü
taşımalar / yazmalar / yer değiştirmeler	her dizi yazması	kayıtlar büyük, ya da yazmalar pahalı

Kararlılık, tanımlanmış

- Eşit anahtarlar çıktıda **girdi sırasını** korur
- Doğrulukla ilgili değil — çıktı yine de sıralıdır
- Bazı algoritmaların ücretsiz verdiği **ek** bir garanti
- Bölüm 11'de özel bir animasyon alır

Yerinde mi, ek bellek mi?

- **Yerinde:** $O(1)$ ek bellekle yeniden düzenler
- Kabarcık, seçmeli, eklemeli, shell, hızlı sıralama: yerinde
- Birleştirmeli sıralama $O(n)$ ek ister — garantisinin bedeli
- Sayma/radix sıralaması da $O(n+k)$ ek ister

2. Kabarcık Sıralaması

Başlangıç sorusu

İki komşuyu karşılaştırıp ters sıradaysa yer değiştirmek en doğal sıralama içgüdüsüdür. Bu fikir tek başına sizi ne kadar ileri götürür?

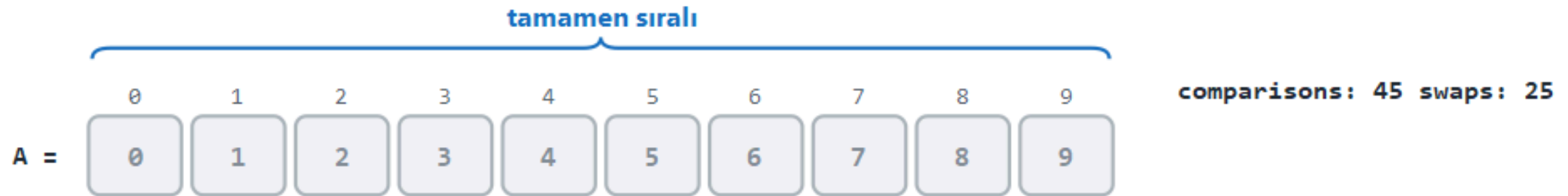
Kısa bir tarihçe

- Hızlı ya da birleştirmeli sıralamanın aksine tek bir adı geçen mucidi yok
- Bilgisayarlıktaki en eski sıralama fikirlerinden biri
- Literatürde 1950'lerin ortasından itibaren görünür
- İcat etmek için hiç zekâ gerektirmediği için ilk öğretilir

Kabarcık sıralaması fikri

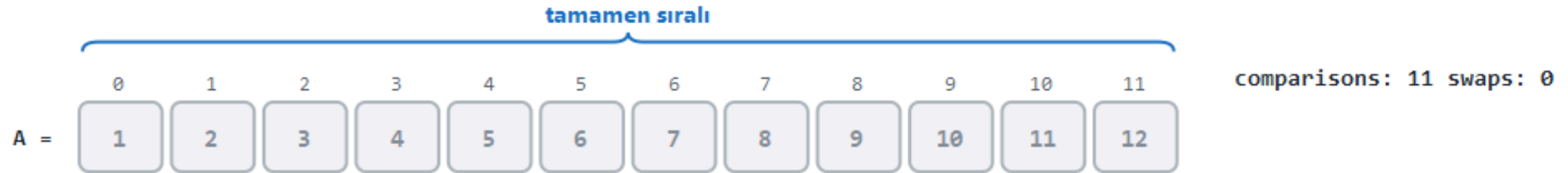
- Soldan sağa yürü, her komşu çifti karşılaştı
- Sırası yanlışsa yer değiştir — büyük değer sağa "kabarcıklanır"
- Tüm yürüyüşü tekrarla, her seferinde bir hücre daha kısa
- **Erken çıkış:** sıfır yer değiştirmeli bir tur bitti demek

Kabarcık sıralaması, adım adım



Bitti: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]. Toplam 45 karşılaştırma, 25 yer değiştirme. En kötü durumda $O(n^2)$, erken çıkış sayesinde en iyi durumda (zaten sıralı) $O(n)$.

Uç durum — zaten sıralı



Bitti: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]. Toplam 11 karşılaştırma, 0 yer değiştirme. En kötü durumda $O(n^2)$, erken çıkış sayesinde en iyi durumda (zaten sıralı) $O(n)$.

Kod — tur ve erken çıkış

```
for (int pass = 0; pass < n - 1; pass++) {  
    int swapped = 0;  
    for (int i = 0; i < n - 1 - pass; i++) {  
        if (a[i] > a[i + 1]) {  
            int tmp = a[i]; a[i] = a[i + 1];  
            a[i + 1] = tmp; swapped = 1;  
        }  
    }  
    if (!swapped) break;  
}
```

Karmaşıklık ve hatalar

- En iyi durum $O(n)$: zaten sıralı girdide erken çıkış
- En kötü/ortalama durum $O(n^2)$: tersten sıralı ya da rastgele
- Sık hata: `>=` kullanmak, kararlılığı bozar
- Sık hata: erken-çıkış bayrağını tamamen unutmak

Mini soru

Kabarcık sıralaması neden **kararlı** — hangi belirli kod satırı, iki eşit elemanın asla birbirinin üzerinden geçirilmediğini garanti eder?

Yanıt

`if (a[i] > a[i+1])` 'deki kesin `>` .

Eşit elemanlar asla `>` 'yi sağlamaz, bu yüzden asla yer değiştirmezler — girdi sıraları korunur.

3. Seçmeli Sıralama

Başlangıç sorusu

Kabarcık sıralamasının tersten sıralı durumu 12 değeri sıralamak için 66 yer değiştirme yaptı. Gerçek minimumu bir kez tarayıp doğrudan yerleştirseniz?

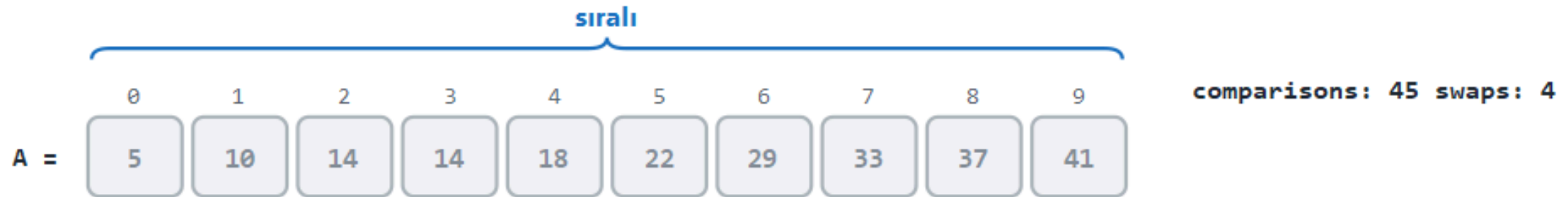
Kısa bir tarihçe

- Kabarcık sıralaması gibi, tek bir adı geçen mucidi yok
- "Kalan en küçüğü seç" fikrinin doğrudan algoritmik biçimi
- En az sıralamanın kendisi kadar eski
- Bugün minimal-yazma garantisi için değerli

Seçmeli sıralama fikri

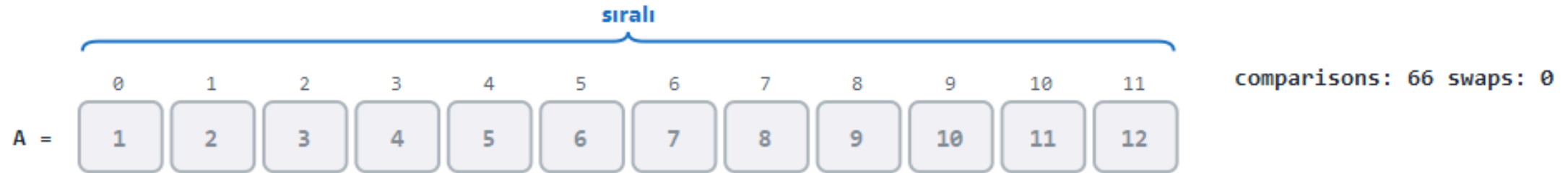
- Her i için, $i+1..n-1$ 'i minimumun indisi için tara
- i konumuna yer değiştir — zaten oradaysa atla
- Sıralı bölge **soldan** büyür (kabarcık: sağdan)
- En çok $n-1$ yer değiştirme, hiç — ama her zaman $O(n^2)$ tarama

Seçmeli sıralama, adım adım



Bitti: [5, 10, 14, 14, 18, 22, 29, 33, 37, 41]. Toplam 45 karşılaştırma, yalnızca 4 yer değiştirme. Her durumda $O(n^2)$ karşılaştırma yapar (erken çıkış yok) ama en çok $n-1$ yer değiştirme -- yazma maliyeti pahalıysa avantajlıdır.

Uç durum — zaten sıralı



Bitti: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]. Toplam 66 karşılaştırma, yalnızca 0 yer değiştirme. Her durumda $O(n^2)$ karşılaştırma yapar (erken çıkış yok) ama en çok $n-1$ yer değiştirme -- yazma maliyeti pahalıysa avantajlıdır.

Kod — minimumu bul, bir kez yer değiştir

```
for (int i = 0; i < n - 1; i++) {  
    int min_idx = i;  
    for (int j = i + 1; j < n; j++)  
        if (a[j] < a[min_idx]) min_idx = j;  
    if (min_idx != i) {  
        int tmp = a[i];  
        a[i] = a[min_idx]; a[min_idx] = tmp;  
    }  
}
```

Karmaşıklık ve hatalar

- En iyi/ortalama/en kötü: hepsi her zaman $O(n^2)$ karşılaştırma
- En çok $n-1$ yer değiştirme — bugünkü herhangi bir sıralamanın en azı
- Sık hata: koşulsuz yer değiştirmek, yazma israfı
- **Kararlı değil**: uzun-mesafeli yer değiştirme bir bağı sıçratabilir

Mini soru

Seçmeli sıralama, girdi ne olursa olsun en çok $n-1$ yer değiştirme yapar. Bu sınır dizi ne kadar karışık olursa olsun neden geçerlidir?

Yanıt

Dış-döngü adımı başına en çok bir yer değiştirme.

Hareket etmesi gereken her eleman bir kez, doğrudan son yerine hareket eder — $n-1$ adım, en çok $n-1$ yer değiştirme.

4. Eklemeli Sıralama

Başlangıç sorusu

Bir deste oyun kartını sıralamayı düşünün.
Hiçbir zaman her kartı her kartla karşılaştırmazsınız.
Onun yerine gerçekte ne yaparsınız?

Kısa bir tarihçe

- Kart-sıralama fikrinin kendisi — tek bir mucidi yok
- En eski, en sezgisel sıralama yöntemlerinden biri
- Öğretmenin kendi tahta çizimleri tam olarak bu modeli kullanır
- Gerçek kütüphane sıralamaları küçük alt diziler için ona döner

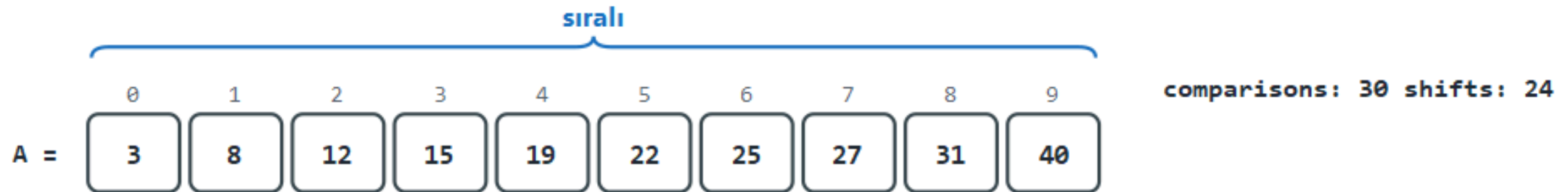
Eklemeli sıralama fikri

- $A[0..i-1]$ her zaman **zaten sıralı**
- $A[i]$ 'yi **anahtar** (kırmızı kutu) olarak dışarı çek
- Her $> \text{key}$ elemanı bir hücre sağa kaydır
- Anahtarı açtığı boşluğa bırak

Tahta-çizimi modeli

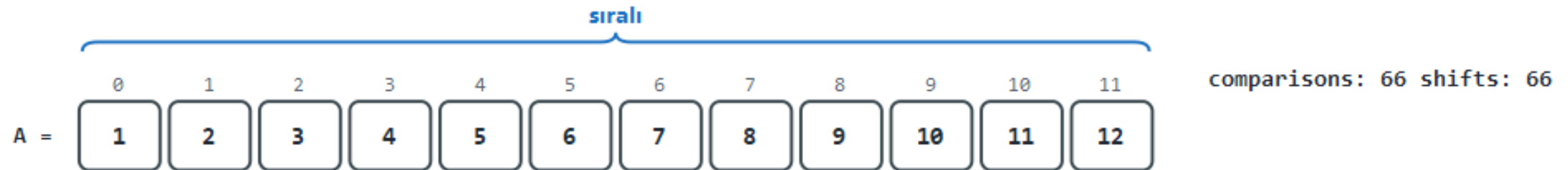
- $A[0..i-1]$ üzerinde **parantez**: "zaten sıralı"
- **Kırmızı kutu**: anahtar, diziden dışarı çekilmiş
- $\leq$ **key** bölgesi kalır; $>$ **key** bölgesi kayar
- **Kaydırma oku**: boşluğun bir adım sola hareketi

Eklemeli sıralama, adım adım



Bitti: [3, 8, 12, 15, 19, 22, 25, 27, 31, 40]. Toplam 30 karşılaştırma, 24 kaydırma. En iyi durum (zaten sıralı) $O(n)$; en kötü durum (tersten sıralı) $O(n^2)$. Kararlıdır (stable).

Uç durum — tersten sıralı



Bitti: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]. Toplam 66 karşılaştırma, 66 kaydırma. En iyi durum (zaten sıralı) $O(n)$; en kötü durum (tersten sıralı) $O(n^2)$. Kararlıdır (stable).

Kod — anahtarı çek, kaydır, bırak

```
for (int i = 1; i < n; i++) {  
    int key = a[i];  
    int j = i - 1;  
    while (j >= 0 && a[j] > key) {  
        a[j + 1] = a[j];  
        j--;  
    }  
    a[j + 1] = key;  
}
```

Karmaşıklık ve hatalar

- En iyi durum $O(n)$: zaten sıralı, i başına bir kontrol
- En kötü durum $O(n^2)$: tersten sıralı, tam kaydırmalar
- Sık hata: $\geq$ yerine $>$ kararlılığı bozar
- Sık hata: $a[j] > \text{key}$ 'i $j \geq 0$ 'dan önce kontrol etmek

Mini soru

Eklemeli sıralamanın en iyi durumu neden tam olarak $O(n)$ 'dir, ve hangi belirli gerçek-dünya veri türü bu en iyi durumu yaygın durum yapar?

Yanıt

Zaten sıralı (ya da neredeyse öyle) girdi.

Her $a[j] \leq \text{key}$ kontrolü hemen başarılı olur,
bu yüzden n dış adımının çoğu yalnızca $O(1)$ 'e mal olur.

5. Shell Sıralaması

Başlangıç sorusu

Eklemeli sıralama tam olarak bir eleman evinden uzakta başladığında yavaştır. Önce o mesafenin çoğunu büyük sıçramalarla kapatsanız?

Kısa bir tarihçe

- Donald L. Shell, "A High-Speed Sorting Procedure"
- *Communications of the ACM*, 1959
- Düz $O(n^2)$ 'yi aşan ilk sıralama algoritması
- Aralık dizisi seçimi hâlâ açık bir araştırma alanı

Shell sıralaması fikri

- Eklemeli sıralama, ama 1 değil `gap` konum uzaklıkta karşılaştırma
- `gap = n/2` ile başla, her turda yarıya indir, `gap = 1` 'de bitir
- Büyük aralıklar uzak elemanları tek sıçramada evine taşır
- `gap = 1` 'de, dizi "neredeyse sıralı" — ucuz bir tur

Shell sıralaması, adım adım

gap = 1

comparisons: 31 shifts: 14

	0	1	2	3	4	5	6	7	8	9
A =	2	5	9	12	17	23	28	33	41	46

Bitti: [2, 5, 9, 12, 17, 23, 28, 33, 41, 46]. Toplam 31 karşılaştırma, 14 kaydırma. Bu aralık dizisiyle en kötü durum $O(n^2)$, ama pratikte adi eklemeli sıralamadan çok daha hızlıdır.

Uç durum — tersten sıralı



Bitti: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]. Toplam 39 karşılaştırma, 24 kaydırma. Bu aralık dizisiyle en kötü durum $O(n^2)$, ama pratikte adi eklemeli sıralamadan çok daha hızlıdır.

Kod — aralıklı eklemeli sıralama

```
for (int gap = n/2; gap > 0; gap /= 2) {  
    for (int i = gap; i < n; i++) {  
        int key = a[i];  
        int j = i;  
        while (j >= gap && a[j-gap] > key) {  
            a[j] = a[j - gap];  
            j -= gap;  
        }  
        a[j] = key;  
    }  
}
```

Karmaşıklık ve hatalar

- Bu basit aralık diziyle en kötü durum $O(n^2)$
- Pratikte düz eklemeli sıralamadan çok daha hızlı
- Daha iyi diziler (Hibbard, Sedgewick) sınırı düşürür
- **Kararlı değil**: aralıklı hareketler eşit elemanları geçebilir

Mini soru

Tersten sıralı örnekte, en küçük değer son indiste başlar. Üç aralık turu boyunca toplam kaç kaydırma gerekir?

Yanıt

Toplam dört kaydırma (büyüklükleri 6, 3, 1, 1) —
düz eklemeli sıralamanın 11 tek-hücrelik kaydırmaya
ihtiyaç duyacağı aynı 11-konumluk mesafeyi kaplayarak.

6. Birleştirmeli Sıralama

Başlangıç sorusu

Elemanları tek bir dizinin içinde hareket ettirmek yerine, onu ikiye bölseydiniz, her yarıyı tamamen sıralasaydınız, sonra birleştirseydiniz?

Kısa bir tarihçe

- **John von Neumann**, 1945
- Saklı-program bir bilgisayar için ilk algoritmalarından biri
- Bilgisayarlıkta **böl ve yönetin** kurucu örneği
- Örüntü bu dersin geri kalanında sürekli tekrar karşınıza çıkar

Böl-ve-yönet fikri

- $[lo, hi)$ 'yi orta noktasından ikiye böl
- Her yarıyı aynı şekilde özyinelemeli olarak sırala
- İki sıralı yarıyı bir yardımcı diziyle **birleştir**
- $\log_2(n)$ seviye $\times$ seviye başına $O(n)$ = $O(n \log n)$

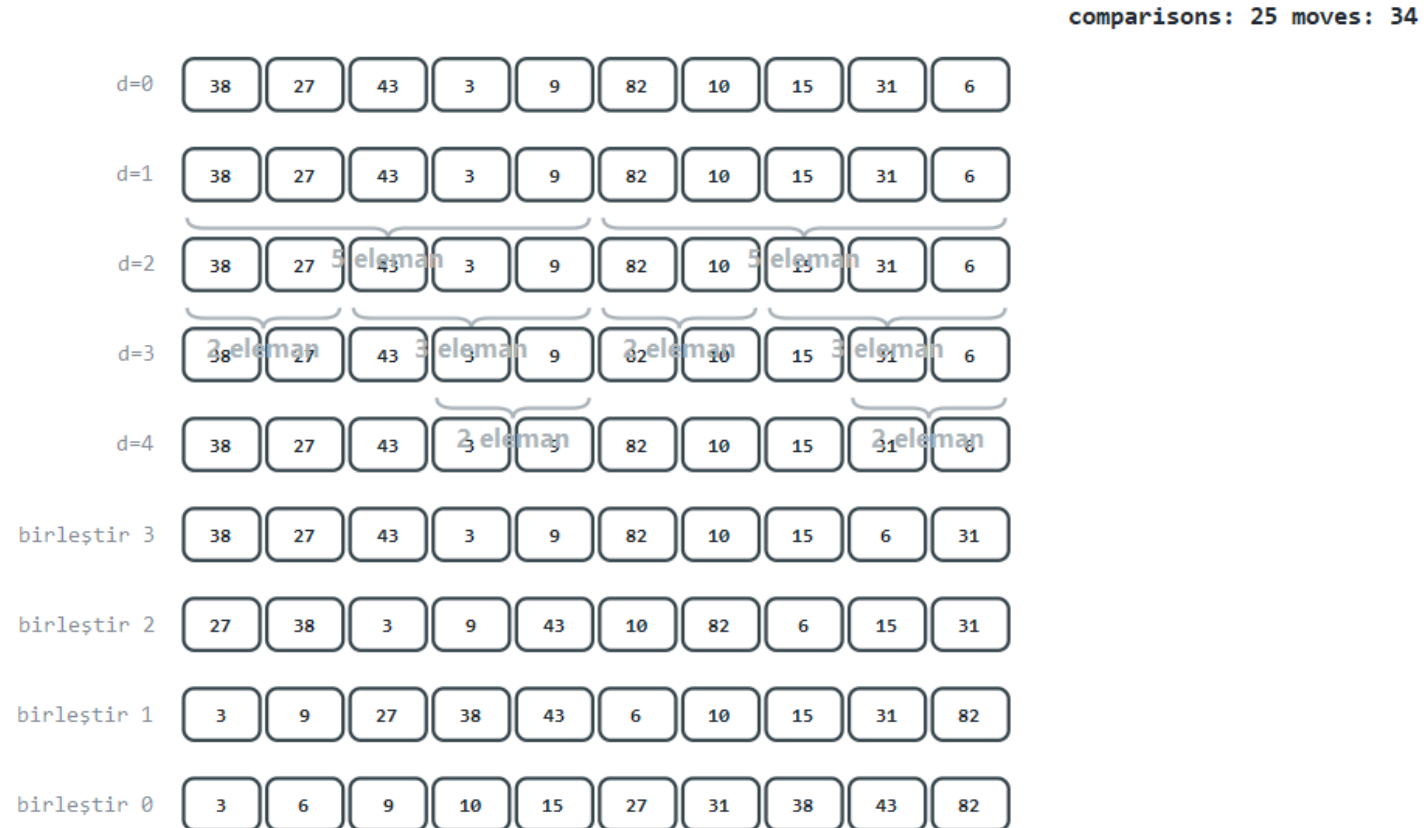
Garanti neden hiç bozulmaz

- Hiçbir yerde erken çıkış, veriye bağlı kısayol yok
- Her seviye her zaman her elemana bir kez dokunur
- Bedel: n büyüklüğünde bir yardımcı dizi — $O(n)$ ek
- İki eşdeğer biçim: yukarıdan aşağı, aşağıdan yukarı

Yukarıdan aşağı — satır olarak özyineleme derinliği

- $d=0$: tüm dizi; $d=1$: iki yarı; ...
- Bölme değerleri hiç değiştirmez, yalnızca parantez sınırlarını
- Sonra birleştirme aynı satırlarda geri yukarı çıkar
- Son satır: tamamen sıralı dizi

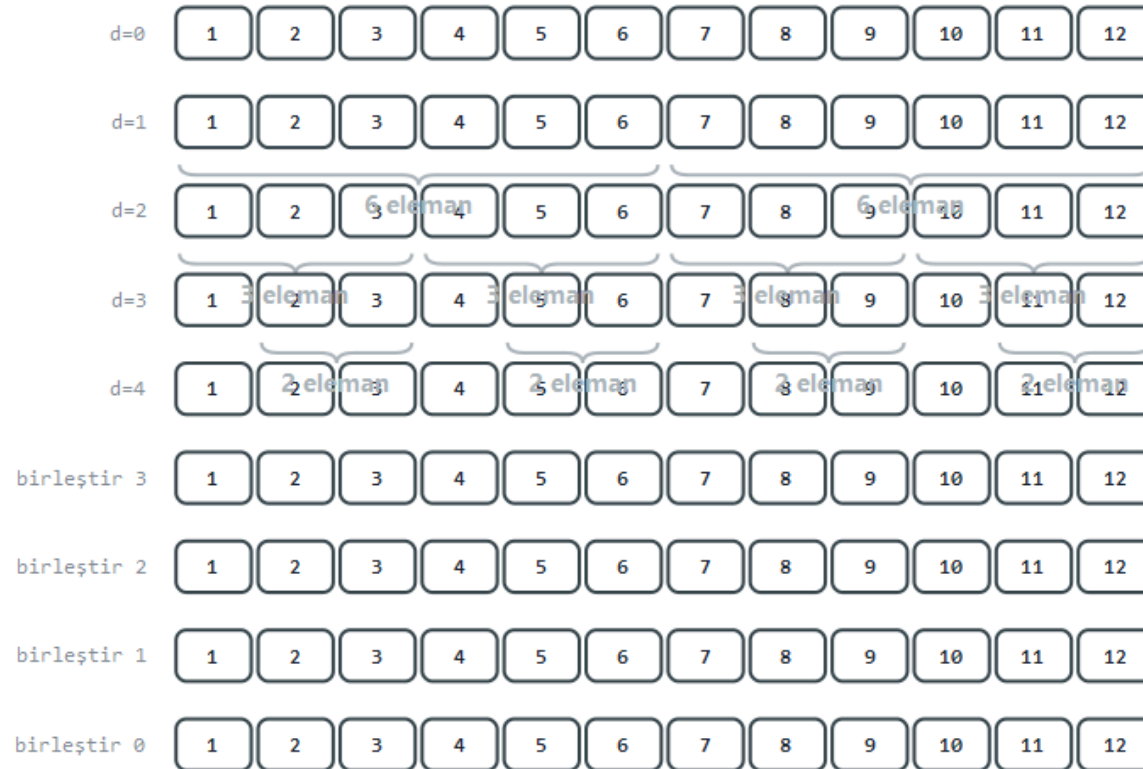
Birleştirmeli sıralama (yukarıdan aşağı), adım adım



Bitti: [3, 6, 9, 10, 15, 27, 31, 38, 43, 82]. Toplam 25 karşılaştırma, 34 yazma. Her durumda $O(n \log n)$; ekstra tmp dizisi yüzünden $O(n)$ ek bellek kullanır ama kararlıdır (stable).

Uç durum — zaten sıralı

comparisons: 20 moves: 44



Bitti: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]. Toplam 20 karşılaştırma, 44 yazma. Her durumda $O(n \log n)$; ekstra tmp dizisi yüzünden $O(n)$ ek bellek kullanır ama kararlıdır (stable).

Kod — birleştirme adımı

```
int i = lo, j = mid, k = lo;
while (i < mid && j < hi)
    tmp[k++] = (a[i] <= a[j])
                ? a[i++] : a[j++];
while (i < mid) tmp[k++] = a[i++];
while (j < hi)  tmp[k++] = a[j++];
for (int x=lo; x<hi; x++) a[x]=tmp[x];
```

Kod — özyinelemeli sürücü

```
void merge_sort(int a[], int lo,
                int hi, int tmp[]) {
    if (hi - lo <= 1) return;
    int mid = lo + (hi - lo) / 2;
    merge_sort(a, lo, mid, tmp);
    merge_sort(a, mid, hi, tmp);
    merge(a, lo, mid, hi, tmp);
}
```

Aşağıdan yukarı — hiç özyineleme yok

- Her eleman genişliği 1 olan sıralı bir çalışma olarak başlar
- Bitişik çalışmaları genişlik-2'ye, sonra genişlik-4'e birleştir
- Katlama bir çalışma tüm diziyi kapladığında durur
- Aynı `merge()`, döngüyle değil çağrı yığınıyla ulaşılır

Birleştirmeli sıralama (aşağıdan yukarı), adım adım

comparisons: 25 moves: 36



Bitti: [3, 6, 9, 10, 15, 27, 31, 38, 43, 82]. Toplam 25 karşılaştırma, 36 yazma. Aynı $O(n \log n)$ maliyet, ama özyineleme çağrı yığını (call stack) hiç kullanılmaz.

Uç durum — n bir 2 kuvveti

comparisons: 47 moves: 64



Bitti: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16]. Toplam 47 karşılaştırma, 64 yazma. Aynı $O(n \log n)$ maliyet, ama özyineleme çağrı yığını (call stack) hiç kullanılmaz.

Kod — genişlik-katlama döngüsü

```
for (int width=1; width<n; width*=2) {  
    for (int lo=0; lo<n-width;  
        lo += 2*width) {  
        int mid = lo + width;  
        int hi = min(mid+width, n);  
        merge(a, lo, mid, hi, tmp);  
    }  
}
```

Karmaşıklık ve hatalar

- Her durumda $O(n \log n)$ — en iyi, ortalama, en kötü
- $O(n)$ ek bellek — o garantinin bedeli
- **Kararlı:** $\leq$ eşitlik kırılımı her zaman sol çalışmayı tercih eder
- Sık hata: $<$ yerine $\leq$ sessizce kararlılığı bozar

Mini soru

Hem yukarıdan aşağı hem aşağıdan yukarı
birleştirmeli sıralama aynı toplam işi yapar.

Aşağıdan yukarının bir avantajını, yukarıdan aşağının bir avantajını adlandırın.

Yanıt

Aşağıdan yukarı: çağrı yığınını hiç kullanmaz, onu taşıramaz. **Yukarıdan aşağı:** özyinelemeli yapısı uyarlamak daha kolaydır — örn. erken eklemeli sıralamaya geçiş.

7. Hızlı Sıralama

Başlangıç sorusu

Birleştirmeli sıralamanın $O(n \log n)$ 'i $O(n)$ ek belleğe mal olur. Ortalamada aynı derecede hızlı ama yerinde sıralayan böl-ve-yönet bir sıralama var mı?

Kısa bir tarihçe

- **C. A. R. Hoare**, 1959–1960, Moskova Devlet Üniversitesi
- "Quicksort" olarak yayımlandı, *The Computer Journal*, 1962
- Hâlâ birçok dil kütüphanesinin varsayılan dizi sıralaması
- Bu derste en önemli sonuçlara sahip algoritmalardan biri

Hızlı sıralama fikri

- Aralığın bir elemanını **pivot** olarak seç
- **Bölümle:** `<= pivot` sola, `>= pivot` sağa
- Sol parçayı ve sağ parçayı özyinelemeli olarak hızlı-sırala
- Ayrı bir "birleştir" adımı yok — bölümleme işi yapar

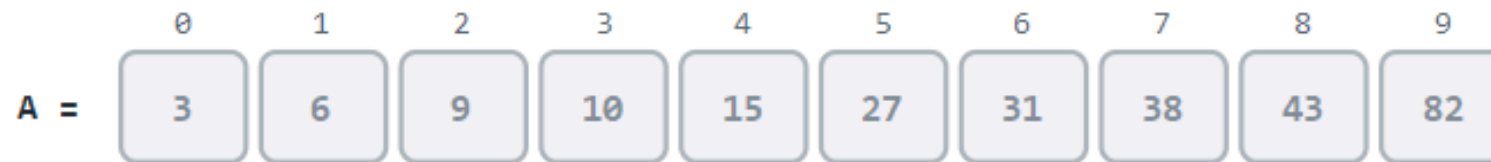
İki klasik bölümlleme şeması

- **Lomuto**: pivot = son eleman, bir ileri tarama
- **Hoare**: pivot = ilk eleman, iki içeri tarama
- İkisi de yan yana incelenir — neredeyse herkesi
- ilk seferinde şaşırtan bir biçimde farklılaşırlar

Lomuto bölümlleme — fikir

- Pivot = aralığın **son** elemanı
- i , " $\leq \text{pivot}$ " bölgesinin sınırını işaretler
- j soldan sağa tarar; $a[j] \leq \text{pivot}$ i 'yi ilerletir
- Pivot $i+1$ 'e kayar — nihai, doğru konumu

Hızlı sıralama (Lomuto), adım adım

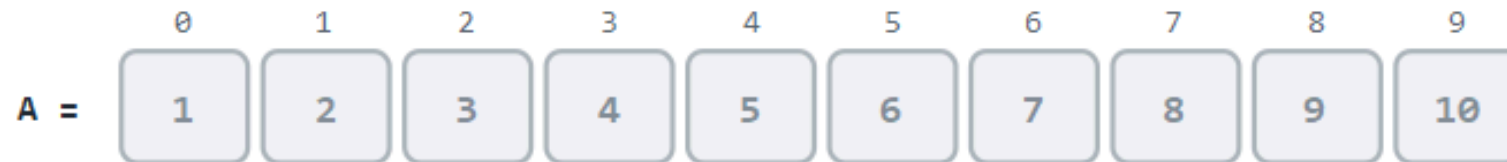


comparisons: 25 swaps: 11

Bitti: [3, 6, 9, 10, 15, 27, 31, 38, 43, 82]. Toplam 25 karşılaştırma, 11 yer değiştirme. Ortalama $O(n \log n)$; zaten sıralı/tersten sıralı girdide $O(n^2)$ (bkz. quick-sort-worst-case).

Uç durum — zaten sıralı

comparisons: 45 swaps: 9



Bitti: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]. Toplam 45 karşılaştırma, 9 yer değiştirme. Ortalama $O(n \log n)$; zaten sıralı/tersten sıralı girdide $O(n^2)$ (bkz. quick-sort-worst-case).

Kod — Lomuto bölümlleme

```
int pivot = a[hi];
int i = lo - 1;
for (int j = lo; j < hi; j++) {
    if (a[j] <= pivot) {
        i++;
        int t=a[i]; a[i]=a[j]; a[j]=t;
    }
}
int t=a[i+1]; a[i+1]=a[hi]; a[hi]=t;
return i + 1;
```

Kod — Lomuto'nun özyinelemesi

```
void quick_sort_lomuto(  
    int a[], int lo, int hi) {  
    if (lo < hi) {  
        int p = partition_lomuto(  
            a, lo, hi);  
        quick_sort_lomuto(a, lo, p-1);  
        quick_sort_lomuto(a, p+1, hi);  
    }  
}
```

Hoare bölümlleme — fikir

- Pivot = aralığın **ilk** elemanı
- İki işaretçi **i**, **j** her iki uçtan **içeri** doğru tarar
- Sırası yanlış bir çift bulunduğunda yer değiştir; tekrarla
- Pivotun nihai konumunun **j** olduğu **garanti değildir**

Hızlı sıralama (Hoare), adım adım

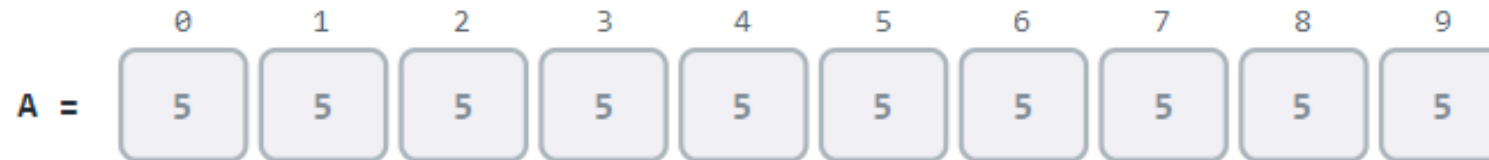
comparisons: 55 swaps: 12

	0	1	2	3	4	5	6	7	8	9
A =	3	6	9	10	15	27	31	38	43	82

Bitti: [3, 6, 9, 10, 15, 27, 31, 38, 43, 82]. Toplam 55 karşılaştırma, 12 yer değiştirme. Hoare, Lomuto'dan genelde daha az yer değiştirme yapar; ortalama $O(n \log n)$.

Uç durum — hepsi eşit değer

comparisons: 48 swaps: 15



Bitti: [5, 5, 5, 5, 5, 5, 5, 5, 5, 5]. Toplam 48 karşılaştırma, 15 yer değiştirme. Hoare, Lomuto'dan genelde daha az yer değiştirme yapar; ortalama $O(n \log n)$.

Kod — Hoare bölümlleme

```
int pivot = a[lo];
int i = lo - 1, j = hi + 1;
while (1) {
    do { i++; } while (a[i] < pivot);
    do { j--; } while (a[j] > pivot);
    if (i >= j) return j;
    int t=a[i]; a[i]=a[j]; a[j]=t;
}
```

Kod — Hoare'ın özyineleme tuzağı

```
void quick_sort_hoare(  
    int a[], int lo, int hi) {  
    if (lo < hi) {  
        int p = partition_hoare(  
            a, lo, hi);  
        quick_sort_hoare(a, lo, p);  
        quick_sort_hoare(a, p+1, hi);  
    }  
}
```

Hızlı sıralamanın en kötü durumu

- Yukarıdaki her iki şema da **sabit bir köşeyi** pivot seçer
- Her zaman en küçük/en büyük eleman = en kötü bölünme
- Sıralı ya da tersten sıralı girdi bunu doğrudan tetikler
- **Üçün-medyanı**, standart gerçek-dünya savunmasıdır

Pivot stratejilerini karşılaştırma

aynı girdi, farklı pivot seçimi

ilk eleman



comparisons: 45 calls: 9

orta indis



comparisons: 19 calls: 6

üçün medyanı



comparisons: 19 calls: 6

Bitti. ilk eleman: 45 karşılaştırma, derinlik 9; orta indis: 19 karşılaştırma, derinlik 3; üçün medyanı: 19 karşılaştırma, derinlik 3. Sabit bir köşe elemanını (ilk ya da son) pivot seçmek, zaten sıralı/tersten sıralı girdide $O(n^2)$ 'ye düşer; orta indis ya da üçün medyanı bu girdide dengeli kalır.

Uç durum — rastgele girdi

aynı girdi, farklı pivot seçimi

ilk eleman



comparisons: 26 calls: 7

orta indis



comparisons: 27 calls: 7

üçün medyanı



comparisons: 21 calls: 6

Bitti. ilk eleman: 26 karşılaştırma, derinlik 6; orta indis: 27 karşılaştırma, derinlik 6; üçün medyanı: 21 karşılaştırma, derinlik 4. Sabit bir köşe elemanını (ilk ya da son) pivot seçmek, zaten sıralı/tersten sıralı girdide $O(n^2)$ 'ye düşer; orta indis ya da üçün medyanı bu girdide dengeli kalır.

Sayılar, somutlaşmış

Strateji	Karşılaştırma (n=14, sıralı)	Derinlik
İlk eleman	91	13
Üçün medyanı	31	3

Karşılaştırmalı-sıralama alt sınırı

- n ayrık elemanın $n!$ olası sıralaması
- Hepsini ayırt eden bir karar ağacının ihtiyacı var
- en az $\log_2(n!)$ seviyeye — ki bu $O(n \log n)$
- Hiçbir karşılaştırmalı sıralama bu sınırı asla aşamaz

Karmaşıklık ve hatalar

- Ortalama $O(n \log n)$, yerinde — pratikte en hızlısı
- Kötü seçilmiş, düşmanca bir pivotta **en kötü $O(n^2)$**
- Sık hata: Lomuto'nun `p-1` 'ini Hoare'ın `p` 'siyle karıştırmak
- **Kararlı değil**: her iki şema da uzun mesafeler boyunca yer değiştirir

Mini soru

Hoare'ın bölümlemesinin neden Lomuto'nunki gibi
(lo , $p-1$) değil (lo , p) ve ($p+1$, hi)
üzerine özyinelemesi gerektiğini bir cümleyle açıklayın.

Yanıt

Hoare pivotun p 'de oturduğunu asla garanti etmez —

yalnızca $\leq p$ olan her şeyin $\leq \text{pivot}$ olduğunu.

p 'yi dışlamak ($p-1$ gibi) sıralanmamış bir elemanı düşürebilir.

8. Sayma Sıralaması

Başlangıç sorusu

$O(n \log n)$ alt sınırı yalnızca iki anahtarın nasıl karşılaştırıldığını bildiğinizi varsayar.

Ya her değer küçük, negatif olmayan bir tam sayı olduğunu biliyorsanız?

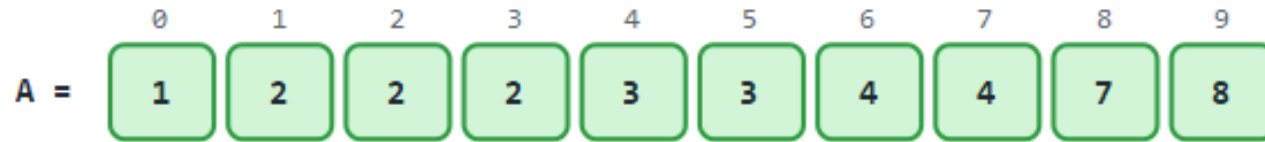
Kısa bir tarihçe

- **Harold H. Seward**, MIT yüksek lisans tezi, 1954
- Karşılaştırma alt sınırını aşmanın en basit yolu
- Anahtar aralığının `0..maxVal` olduğunu önceden bilmeyi gerektirir
- Radix sıralamasının temeli (Bölüm 9)

Sayma sıralaması fikri

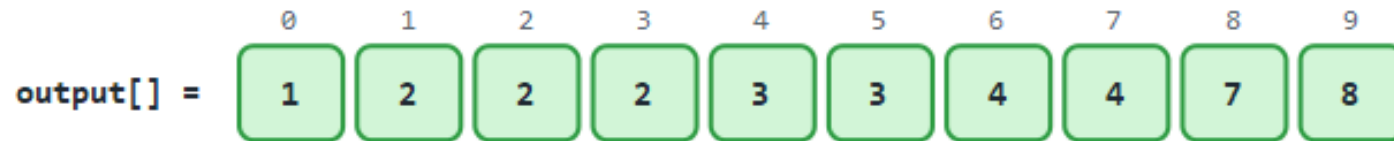
- Her değerin tekrar sayısını `count[]` 'a say
- Sayıları **kümülatif** yap: `count[v]` = "kaç tane `<= v` "
- Her değeri `output[count[a[i]]-1]` 'e yerleştir, azalt
- **Geriye doğru** tara — kararlı tutan şey bu

Sayma sıralaması, adım adım



comparisons: 0 (non-comparison sort)

writes: 20



Bitti: [1, 2, 2, 2, 3, 3, 4, 4, 7, 8]. Toplam 20 yazma, 0 karşılaştırma. $O(n + k)$ zaman ve $O(n + k)$ ek bellek ($k = \text{maxVal}$). Kararlıdır (stable). k , n 'den çok büyükse (bkz. "seyrek aralık") verimsizdir.

Uç durum — seyrek aralık

A =

0	1	2	3	4	5	6	7	8	9
0	1	3	6	7	8	9	12	14	15

comparisons: 0 (non-comparison sort)
writes: 20

count[] =

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	1	2	2	3	3	3	4	5	6	7	7	7	8	8	9

output[] =

0	1	2	3	4	5	6	7	8	9
0	1	3	6	7	8	9	12	14	15

Bitti: [0, 1, 3, 6, 7, 8, 9, 12, 14, 15]. Toplam 20 yazma, 0 karşılaştırma. $O(n + k)$ zaman ve $O(n + k)$ ek bellek ($k = \text{maxVal}$). Kararlıdır (stable). k , n 'den çok büyükse (bkz. "seyrek aralık") verimsizdir.

Kod — say, sonra biriktir

```
int count[max_val + 1] = {0};  
for (int i = 0; i < n; i++)  
    count[a[i]]++;  
for (int v = 1; v <= max_val; v++)  
    count[v] += count[v - 1];
```

Kod — geriye tarayarak yerleştir

```
for (int i = n - 1; i >= 0; i--) {  
    output[count[a[i]] - 1] = a[i];  
    count[a[i]]--;  
}  
for (int i = 0; i < n; i++)  
    a[i] = output[i];
```

Karmaşıklık ve hatalar

- $O(n + k)$ zaman ve alan, $k = \text{maxVal}$ — her durumda
- k , n 'den çok büyük değilse $O(n \log n)$ 'i gerçekten aşar
- k , n 'den çok büyükse israfçı (seyrek örnek)
- Yapı gereği, geriye tarama sayesinde **kararlı**

Mini soru

Eşit iki değer $a[p]$ ve $a[q]$, $p < q$ ile.
Geriye taramayı izleyin — $a[p]$ çıktıda neden
her zaman $a[q]$ 'nun SOLUNDA biter?

Yanıt

`a[q]` önce yerleştirilir (tarama $n-1$ 'den 0'a gider),
daha sonraki hücreyi talep eder ve `count[v]` 'yi azaltır.
`a[p]` sonra bir sonraki hücreyi talep eder — bir sola.

9. Radix Sıralaması (LSD)

Başlangıç sorusu

Sayma sıralaması küçük bir anahtar aralığı ister.
Ya anahtarlarınız beş basamaklı tam sayılarsa —
çok fazla ayrı değer, ama her biri küçük basamaklardan kurulu?

Kısa bir tarihçe

- **Herman Hollerith**, 1890 ABD nüfus sayımı tabülasyon makineleri
- Mekanik kart sıralayıcıları bir seferde bir sütun işledi
- Sayma sıralamasının kendisinden çok daha eski
- Bugün hâlâ sabit-genişlikli tam sayı/dizi anahtarları için kullanılıyor

Radix sıralaması (LSD) fikri

- **Birler** basamağından başla, **kararlı** bir sayma sıralaması çalıştır
- **Onlara** geç, sonra **yüzler** — her zaman 10 kova
- Her geçişin kararlılığı isteğe bağlı değil — zorunlu
- `max_val / place` sıfıra ulaştığında durur

Radix sıralaması (LSD), adım adım

A =

0	1	2	3	4	5	6	7	8	9
8	21	100	329	355	436	457	657	720	839

place = 100

comparisons: 0 (non-comparison sort)

writes: 60

count[] =

0	1	2	3	4	5	6	7	8	9
0	0	0	0	0	0	0	0	0	0

output[] =

0	1	2	3	4	5	6	7	8	9

Bitti: [8, 21, 100, 329, 355, 436, 457, 657, 720, 839]. Toplam 60 yazma, 0 karşılaştırma. d = basamak sayısı olmak üzere $O(d * (n + 10))$ zaman; her geçiş kararlı olduğu için sonuç doğru.

Uç durum — tek basamaklı değerler

A =

0	1	2	3	4	5	6	7	8	9
0	1	2	3	4	5	6	7	8	9

place = 1

comparisons: 0 (non-comparison sort)

writes: 20

count[] =

0	1	2	3	4	5	6	7	8	9
0	0	0	0	0	0	0	0	0	0

output[] =

0	1	2	3	4	5	6	7	8	9

Bitti: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]. Toplam 20 yazma, 0 karşılaştırma. d = basamak sayısı olmak üzere $O(d * (n + 10))$ zaman; her geçiş kararlı olduğu için sonuç doğru.

Kod — bir basamak çıkarma

```
int get_digit(int x, int place) {  
    return (x / place) % 10;  
}  
  
for (int place = 1;  
     max_val / place > 0;  
     place *= 10) { ... }
```

Kod — bir basamak geçişi

```
/* kararlı sayma sıralaması,  
   get_digit(a[i], place) anahtarlı */  
count_and_accumulate(place);  
for (int i = n-1; i >= 0; i--)  
    place_by_digit(a[i], place);  
copy_output_back();
```

Karmaşıklık ve hatalar

- $O(d \times (n + b))$ — d basamak, taban $b=10$
- Sabit genişlikli anahtarlar: etkin olarak $O(n)$, log çarpanı yok
- Sık hata: kararsız bir basamak geçişi doğruluğu bozar
- Sık hata: EN anlamlı basamaktan önce sıralamak

Mini soru

Önce en az anlamlı basamağa göre sıralamak neden çalışır, aynı algoritmayı önce en anlamlı basamaktan çalıştırmak neden çalışmaz?

Yanıt

Kararlılık yukarı doğru bileşir. Her geçiş tüm önceki, daha az anlamlı basamaklardan sırayı korur — MSD-ilk üzerine inşa edecek böyle bir garantiye sahip değildir.

10. Kova Sıralaması

Başlangıç sorusu

Ya kesin sayılar yerine, her değeri kabaca büyüklüğüne göre birkaç kovadan birine atıp, sonra her küçük kovayı temizleseniz?

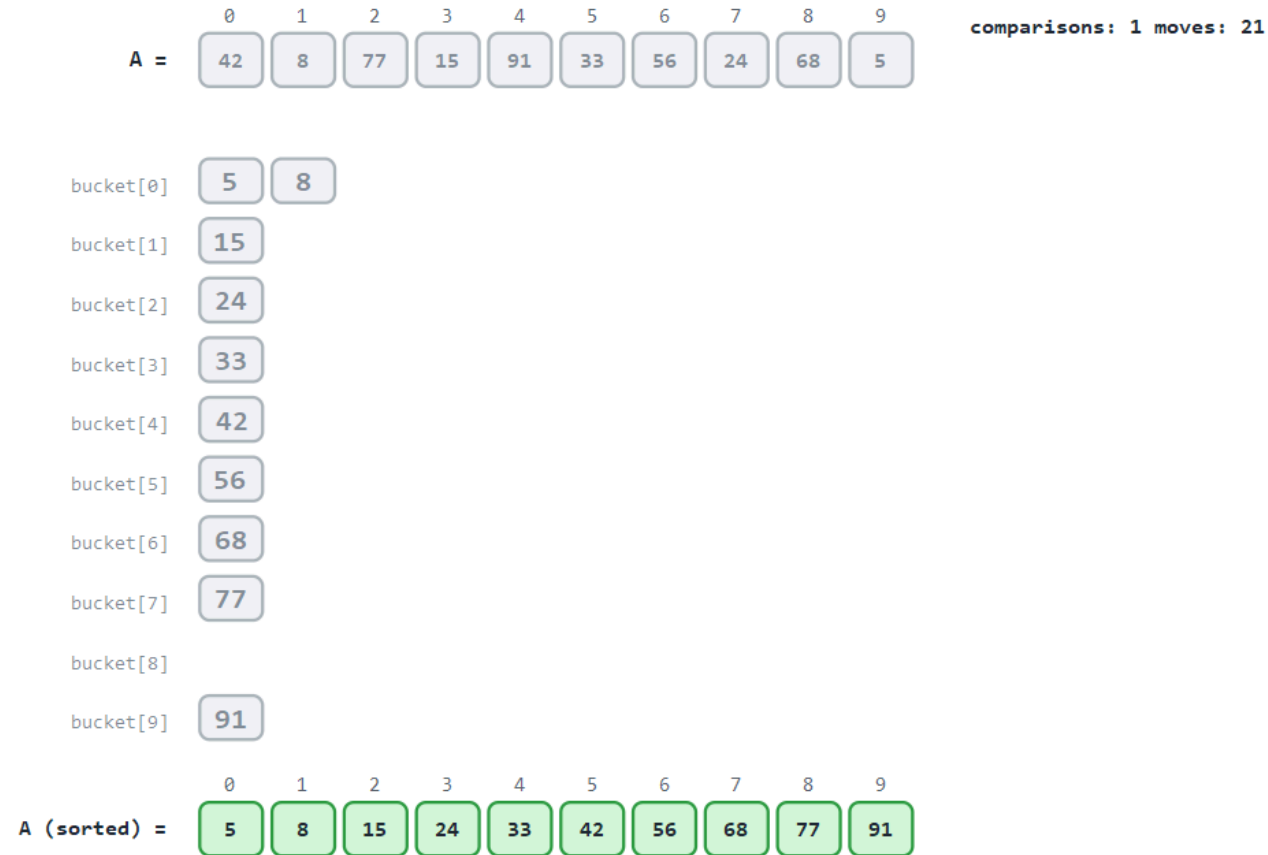
Kısa bir tarihçe

- Bugünkü üç karşılaştırmasız sıralamanın en eskisi ve en geneli
- Tek bir mucidi ya da tarihi yok
- Sayma sıralamasının doğal bir genellemesi
- Tamamen girdinin gerçek dağılımına bağlı

Kova sıralaması fikri

- $[0, \text{max_val}]$ 'i 10 eşit büyüklükte kovaya böl
- Her değeri kovasına dağıt (burada onlar basamağı)
- Her küçük kovayı sırala (eklemeli sıralama — ucuz)
- Kovaları $0..9$ birleştir — otomatik olarak sıralı

Kova sıralaması, adım adım



Bitti: [5, 8, 15, 24, 33, 42, 56, 68, 77, 91]. Toplam 1 karşılaştırma (kova-içi eklemeli sıralamadan), 21 taşıma. Değerler düzgün dağılmışsa ortalama $O(n)$; hepsi tek kovaya düşerse $O(n^2)$.

Uç durum — hepsi tek kovada

A =

0	1	2	3	4	5	6	7	8	9
40	41	42	43	44	45	46	47	48	49

 comparisons: 9 moves: 20

bucket[0]

bucket[1]

bucket[2]

bucket[3]

bucket[4]

40	41	42	43	44	45	46	47	48	49
----	----	----	----	----	----	----	----	----	----

bucket[5]

bucket[6]

bucket[7]

bucket[8]

bucket[9]

A (sorted) =

0	1	2	3	4	5	6	7	8	9
40	41	42	43	44	45	46	47	48	49

Bitti: [40, 41, 42, 43, 44, 45, 46, 47, 48, 49]. Toplam 9 karşılaştırma (kova-içi eklemeli sıralamadan), 20 taşıma. Değerler düzgün dağılmışsa ortalama $O(n)$; hepsi tek kovaya düşerse $O(n^2)$.

Kod — dağıt, sonra bitir

```
int b = (a[i]*BUCKETS)
      / (max_val+1);
bucket[b][bucket_len[b]++]=a[i];
/* ... sonra her kovayı eklemeli-
   sırala, sonra birleştir */
```

Karmaşıklık ve hatalar

- **Ortalama $O(n)$** : değerler kovalar boyunca eşit dağılmış
- **En kötü $O(n^2)$** : tüm değerler tek bir kovaya çarpışır
- Sık hata: gerçek veriyi yok sayan kova sınırları
- İçsel kova-içi sıralama kararlıysa **kararlı**

Mini soru

Kova sıralaması kova başına içsel bir sıralamaya ihtiyaç duyar; sayma sıralaması değer başına buna benzer bir şeye ihtiyaç duymaz. Bunu açıklayan yapısal fark nedir?

Yanıt

Bir kova bir değer **ARALIĞINI** kapsar, bu yüzden hâlâ görelî sıraya ihtiyaç duyan birkaç gerçekten farklı değer tutabilir — sayma sıralamasının sayıları tutamaz.

11. Kararlılık, Gösterilmiş

Başlangıç sorusu

Kararlılığı Bölüm 1'de tanımladık. Kelimeler kanıt değildir. AYNİ girdinin, yalnızca sıralamaya bağlı olarak iki farklı sonuç ürettiğini izleyebilir miyiz?

Gösterinin kurulumu

- Kayıtlar (anahtar, etiket) çiftleri — örn. 5a , 5b , 5c
- Birkaç kayıt kasıtlı olarak bir anahtarı paylaşır ("bağ")
- Satır 1: **eklemeli sıralama** — kararlı (Bölüm 4)
- Satır 2: **seçmeli sıralama** — kararsız (Bölüm 3)

Kararlılık, gösterilmiş



Bitti. Kararsız sıralama: [1a, 2a, 2b, 2c, 5b, 5c, 5a, 8b, 8a, 9a]. Anahtarlar aynı şekilde sıralı, ama en az bir eşit-anahtar grubunda etiket sırası girdiden farklı olabilir -- kararlılık, DOĞRULUK değil, EK bir garantidir.

Uç durum — tüm anahtarlar eşit



Bitti. Kararsız sıralama: [6a, 6b, 6c, 6d, 6e, 6f, 6g, 6h, 6i, 6j]. Anahtarlar aynı şekilde sıralı, ama en az bir eşit-anahtar grubunda etiket sırası girdiden farklı olabilir -- kararlılık, DOĞRULUK değil, EK bir garantidir.

Sonuç, yakından okunmuş

- Kararlı (eklemeli): 5a 5b 5c — tam girdi sırası
- Kararsız (seçmeli): 5b 5c 5a — 5a sona itilmiş
- Bir hata değil — seçmeli sıralamanın uzun-mesafeli yer değiştirmesi iş başında
- Zaten sıralı girdi ikisi arasında HiÇBİR fark göstermez

Hangi sıralamalar kararlı?

Sıralama	Kararlı mı?
Kabarcık, eklemeli, birleştirmeli, sayma, radix	Evet
Seçmeli, shell, hızlı (Lomuto/Hoare)	Hayır
Kova	İçsel sıralama kararlıysa evet

Mini soru

Seçmeli sıralama kararsızdır. Kod olmadan,
onu — ya da herhangi bir karşılaştırmalı
sıralamayı — kararlı hale getirmenin genel bir yolunu betimleyin.

Yanıt

Her elemana özgün indisini ekleyin, ve karşılaştırmada bağları o indise göre kırın. Sıralamanın mekaniğinin hiç değişmesine gerek yok.

12. Algoritmaları Deneysel Olarak Karşılaştırma

Başlangıç sorusu

Big-O sabit çarpanları gizler. "BU verideki gerçek kazananın hangisi olduğunu" çözmenin tek yolu, onları çalıştırıp saymaktır. O halde — sayalım.

Kurulum

- Kabarcık, seçmeli, eklemeli, birleştirmeli, hızlı (Lomuto)
- Hepsi **özdeş** diziyi, her seferinde sıralar
- Her karşılaştırma ve yazma sayılan yardımcılardan geçer
- Toplamlar bir tahmin değil, bir ölçümdür

Sıralama algoritmaları, karşılaştırılıyor

kabarcık	3	6	9	10	15	27	31	38	43	82	comparisons: 45 writes: 52
seçmeli	3	6	9	10	15	27	31	38	43	82	comparisons: 45 writes: 16
eklemeli	3	6	9	10	15	27	31	38	43	82	comparisons: 33 writes: 35
birleştirmeli	3	6	9	10	15	27	31	38	43	82	comparisons: 25 writes: 34
hızlı (Lomuto)	3	6	9	10	15	27	31	38	43	82	comparisons: 25 writes: 34

Karşılaştırma sayısına göre sıralama: 1. birleştirmeli (25) 2. hızlı (Lomuto) (25) 3. eklemeli (33) 4. kabarcık (45) 5. seçmeli (45). Girdi biçimi (zaten sıralı, tersten sıralı, rastgele) hangi algoritmanın kazandığını değiştirebilir -- tek bir "en iyi" algoritma yoktur.

Uç durum — zaten sıralı

kabarcık	1	2	3	4	5	6	7	8	9	10	11	12	comparisons: 11 writes: 0
seçmeli	1	2	3	4	5	6	7	8	9	10	11	12	comparisons: 66 writes: 0
eklemeli	1	2	3	4	5	6	7	8	9	10	11	12	comparisons: 11 writes: 11
birleştirmeli	1	2	3	4	5	6	7	8	9	10	11	12	comparisons: 20 writes: 44
hızlı (Lomuto)	1	2	3	4	5	6	7	8	9	10	11	12	comparisons: 66 writes: 154

Karşılaştırma sayısına göre sıralama: 1. kabarcık (11) 2. eklemeli (11) 3. birleştirmeli (20) 4. seçmeli (66) 5. hızlı (Lomuto) (66). Girdi biçimi (zaten sıralı, tersten sıralı, rastgele) hangi algoritmanın kazandığını değiştirebilir -- tek bir "en iyi" algoritma yoktur.

Dört sayı, bir hikâye

Girdi	Kabarcık	Hızlı (Lomuto)
Zaten sıralı (n=12)	11	66
Tersten sıralı (n=12)	66	66

Tek bir kazanan yok

- Kabarcık: zaten-sıralı veride kesin bir farkla kazanır
- Hızlı (Lomuto): büyük rastgele veride ortalamada kazanır
- Eklemeli: özellikle neredeyse-sıralı veride kazanır
- Kazanan her zaman girdinin **şekline** bağlıdır

Mini soru

Neredeyse-sıralı girdide (yalnızca iki değer yer değiştirmiş), eklemeli sıralama yalnızca 18 karşılaştırmaya, seçmeli tam 66'ya ihtiyaç duydu. Fark neden?

Yanıt

Eklemeli sıralama kısmi sırayı sömürür — çoğu eleman tek bir kontrole durur. Seçmeli sıralamanın tam taraması girdinin ne kadar sıralı olduğunu yok sayar.

13. Bir Sıralama Algoritması Seçmek

Karar tablosu

Durum	En iyi seçim
Çok küçük dizi, ya da özyinelemeli bir alt dizi	Eklemeli sıralama
Zaten / neredeyse sıralı	Eklemeli, ya da kabarcık (erken çıkış)
Yazmalar pahalı, karşılaştırmalar ucuz	Seçmeli sıralama

Karar tablosu, devamı

Durum	En iyi seçim
Garantili $O(n \log n)$ gerekli, bellek uygun	Birleştirmeli sıralama
Ortalama $O(n \log n)$ gerekli, yerinde	Hızlı sıralama, üçün-medyanı
Özyineleme yasak	Birleştirmeli (aşağıdan yukarı), öbek sıralaması

Karar tablosu, sonucu

Durum	En iyi seçim
Küçük tam-sayı anahtarlar, aralık $\approx n$	Sayma sıralaması
Daha büyük tam sayı/dizi anahtarlar	Radix sıralaması (LSD)
Anahtarlar bir aralıkta eşit dağılmış	Kova sıralaması
Bağlar girdi sırasını korumalı	Herhangi bir kararlı sıralama (Bölüm 11.5)

Gerçek soru

- "Genel olarak hangi sıralama en hızlı" değil — böyle bir şey yok
- Verileriniz hakkında **gerçekte ne biliyorsunuz?**
- **Gerçekte neye ihtiyacınız var:** bir garanti mi? kararlılık mı?
- Bölüm 12 size tahmin değil, ölçme aracını verdi

Özet

- $O(n^2)$ ailesi: kabarcık, seçmeli, eklemeli, shell sıralaması
- Garantili $O(n \log n)$: birleştirmeli sıralama (yukarıdan aşağı, aşağıdan yukarı)
- Ortalama $O(n \log n)$, yerinde: hızlı sıralama (Lomuto, Hoare)
- Karşılaştırmasız, $O(n+k)$: sayma, radix, kova sıralaması

Özet, devamı

- Karşılaştırmalı-sıralama alt sınırı: $O(n \log n)$, kanıtlanabilir
- Karşılaştırmасız sıralamalar anahtarları hiç karşılaştırmayarak onu aşar
- Kararlılık: doğruluğun kendisi değil, ek bir garanti
- Tek bir "en iyi" sıralama yok — girdinin şekli karar verir

İleriye bakış

- Hafta 11: **gelişmiş ağaç yapıları**
- Dengeli ağaçlar sıralı sırayı kademeli olarak korur
- Birleştirmeli sıralamanın böl-ve-yönet'i ağaç şeklinde geri döner
- Sonra: **dış sıralama** — bellek için çok büyük veri

Sorular?

CEN207 Veri Yapıları — Hafta 10

Dr. Öğr. Üyesi Uğur CORUH · 2026-2027 Güz